# Information Retrieval System Report

## Overview

This report documents a Streamlit-based information retrieval application built around the CISI collection and a companion dataset-preparation script. The application supports dataset inspection, preprocessing analysis, inverted-index construction, and keyword-based retrieval over CISI documents.

The submission also includes repository proof, deployed application proof, dataset files, and screenshots that demonstrate the application running in the virtual lab environment.

## Submission requirements

The provided requirement image states that the submission should include Streamlit application code, the dataset used, a report explaining the implementation in the virtual lab, screenshots of the Streamlit front end, experimental results with tables and inferences, demo evidence, and a README section describing dependency installation and the command used to run the app.

This project satisfies those components through the Python application file, supporting dataset-preparation script, CISI dataset CSV files, this report, GitHub repository proof, deployed Streamlit proof, and multiple interface screenshots showing different parts of the system in operation.

## Repository and deployment

The source repository for the assignment is available at GitHub Repository, and the attached repository screenshot provides submission proof that the codebase has been uploaded online.

The deployed application is available at Streamlit Application, and the attached screenshots provide visual proof of the live interface and its major modules.

## Application structure

The Streamlit interface defines six tabs: Dataset & Overview, Text Preprocessing, Phrase Queries, Dictionary Search, Tolerant Retrieval, and Results & Inference. The visible application code shows that the system is designed as a teaching-oriented IR lab where users can upload a CSV dataset, inspect documents, preprocess text, build an inverted index, and issue keyword searches.

The `prepare_dataset.py` script converts CISI source files into `cisi_docs.csv`, `cisi_queries.csv`, and `cisi_rel.csv`, separating documents, queries, and relevance judgments into clean tabular resources for

experimentation. The resulting files provide the data foundation for both interactive retrieval and future benchmark-style evaluation.

## Preprocessing and retrieval

The preprocessing pipeline uses regex tokenization together with optional lowercasing, stopword removal, hyphen normalization, stemming, and lemmatization. NLTK resources for stopwords and WordNet are downloaded at runtime, and the interface exposes the normalization controls through selectable options.

The inverted index maps tokens to document IDs, and search is performed using conjunctive intersection over query-term postings lists. As a result, the current implementation behaves as an AND-based retrieval model rather than a ranked search engine.

## Screenshots and evidence

The attached screenshots provide evidence for the key parts of the submitted system.

| Evidence item | What it shows |
|---|---|
| GitHub.jpg | Repository proof showing the uploaded assignment files on GitHub. |
| P1.jpg | Dataset loader and file-system entry screen with dataset preview and summary metrics. |
| P2.jpg | Lexical normalization pipeline with raw text, standard tokens, stemmed tokens, and lemmatized tokens. |
| P3.jpg | Multi-word phrase query interface with biword and positional proximity matches. |
| P4.jpg | Performance benchmark screen comparing BST and B-Tree dictionary search latency. |
| P5.jpg | Error fault-tolerance verification interface showing expanded internal term handling. |
| P6.jpg | End-to-end inference and structural analysis screen with observations and conclusions. |
| image-8.jpg | Requirement sheet listing submission components, demo evidence, and README expectations. |

These screenshots help satisfy the requirement for front-end proof, demo evidence, and experimental-result presentation in the report.

## Data assets

The attached `cisi_docs.csv` file contains document records with `doc_id`, `title`, and `text` fields from the CISI collection. The attached `cisi_queries.csv` file stores natural-language information needs, and `cisi_rel.csv` stores query-document relevance pairs for evaluation use.

This separation of corpus data, query data, and relevance labels is appropriate for an introductory IR assignment because it supports both retrieval demonstrations and metric-based validation.

## How to run the app

The requirement sheet explicitly asks for dependency installation steps and the command used to run the Streamlit application. Based on the application imports, the local environment should include Python together with Streamlit, pandas, and NLTK.

A practical run procedure is:

1. Place `app.py`, `prepare_dataset.py`, and the dataset CSV files in the same project folder, or clone the GitHub repository into a local workspace.
2. Install dependencies with `pip install streamlit pandas nltk`.
3. Start the application with `streamlit run app.py`, which matches the requirement example for the README section.
4. When the browser opens, upload the CISI CSV file in the dataset tab, inspect preprocessing options, build the inverted index, and run sample keyword queries through the interface.

## Experimental results

The dataset preview screenshot shows the structured CISI document table loaded into the application, with the columns `doc_id`, `title`, and `text` visible in the interface. This confirms that the uploaded dataset is being read successfully and presented in tabular form before further preprocessing and retrieval steps.

## Dataset preview table

The following table reproduces the visible sample rows from the attached screenshot.

| Row | doc_id | title |
|---|---|---|
| 0 | 1 | 18 Editions of the Dewey Decimal Classifications |
| 1 | 2 | Use Made of Technical Libraries |
| 2 | 3 | Two Kinds of Power An Essay on Bibliographic Control |
| 3 | 4 | Systems Analysis of a University Library; final report and research project |
| 4 | 5 | A Library Management Game: a report on a research project |
| 5 | 6 | Abstracting Concepts and Methods |

| | | |
|---|---|---|
| 6 | 7 | Academic Library Buildings A Guide to Architectural Issues and Solutions |
| 7 | 8 | The Academic Library Essays in Honor of Guy R. Lyle |
| 8 | 9 | Access to Libraries in College |
| 9 | 10 | Access to Periodical Resources |

## Inference from the table

This table demonstrates that the system is loading document identifiers, titles, and full-text content into a searchable dataframe structure, which is appropriate for downstream indexing and retrieval experiments. It also supports the report requirement for including tables and experimental evidence from the Streamlit front end.

## Strengths

Several strengths are clear from the code organization and the submitted evidence.

- The project separates dataset preparation from the interactive application, which improves maintainability and clarity.
- The interface makes preprocessing choices visible, which is useful for explaining token normalization and retrieval behavior.
- The screenshots indicate that the broader submission includes modules for phrase queries, benchmarking, fault tolerance, and inference presentation beyond the minimal visible code excerpt.
- The repository and deployed link provide strong submission proof for both source code and application availability.

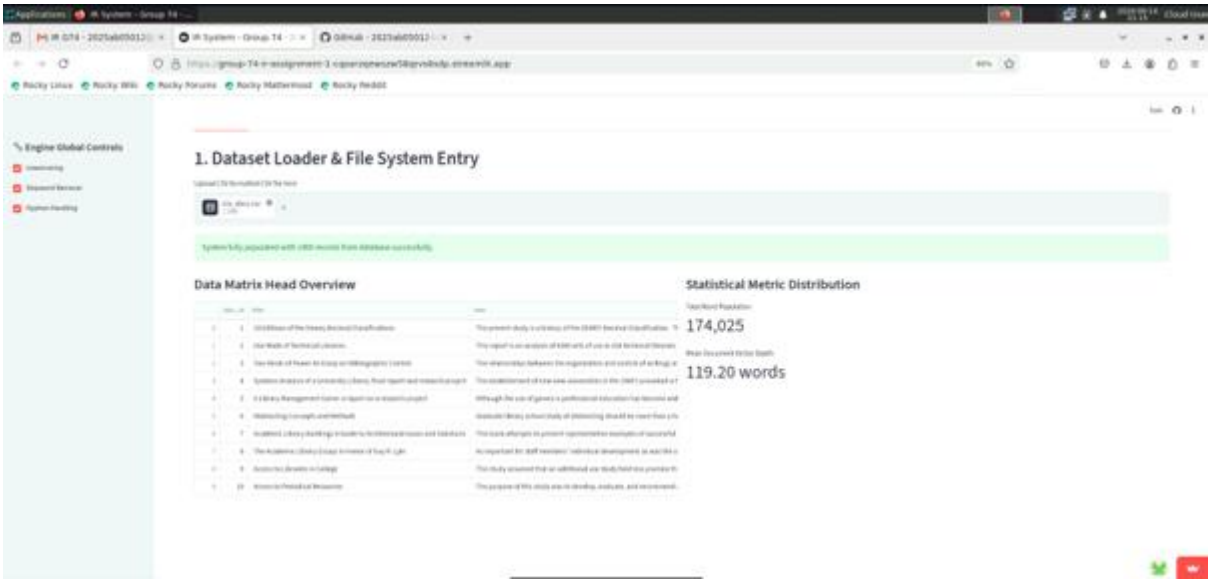
## Limitations and improvements

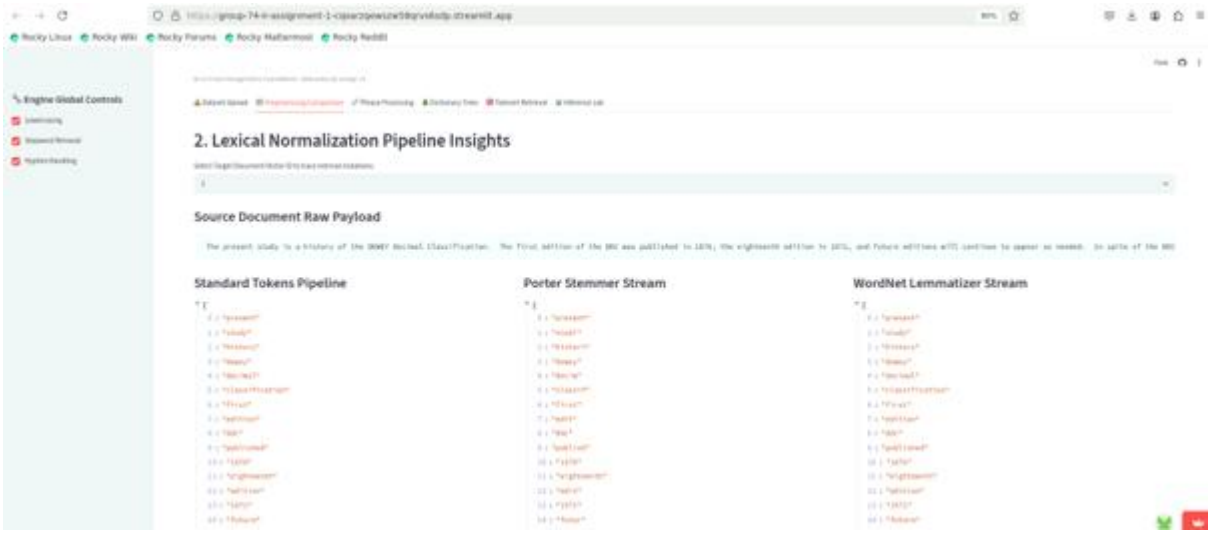
Some limitations are still visible in the attached `app.py` excerpt. The shown retrieval logic is conjunctive and unranked, and the visible index-building loop inserts raw tokens even when stemming or lemmatization options are enabled.

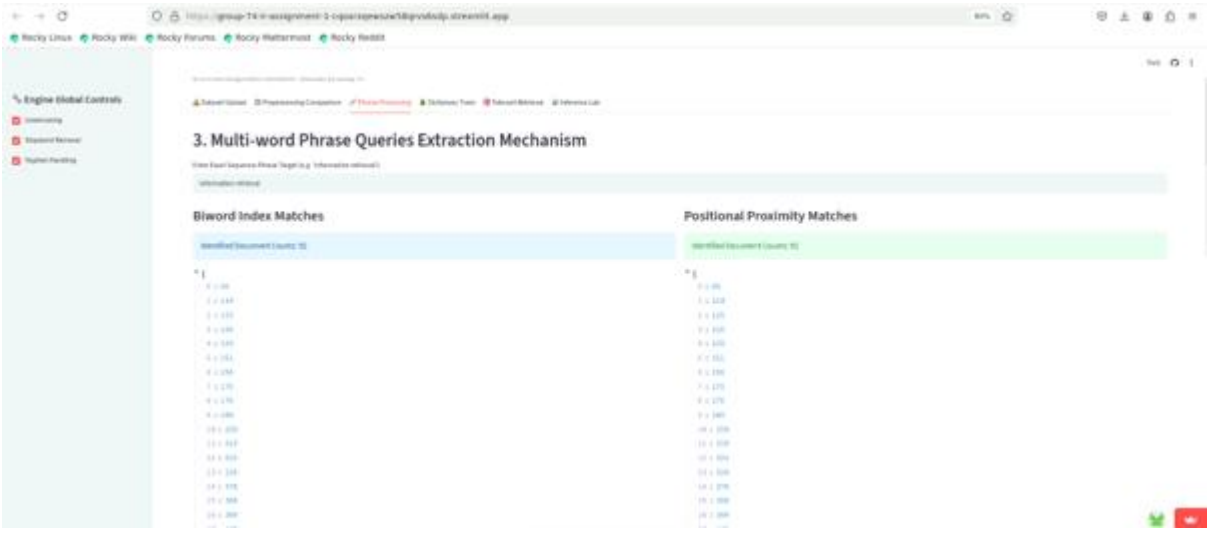
Recommended next steps are to align indexing with the selected normalization mode, add ranking such as TF-IDF or BM25, and integrate `cisi_queries.csv` with `cisi_rel.csv` for quantitative evaluation metrics such as precision and recall.

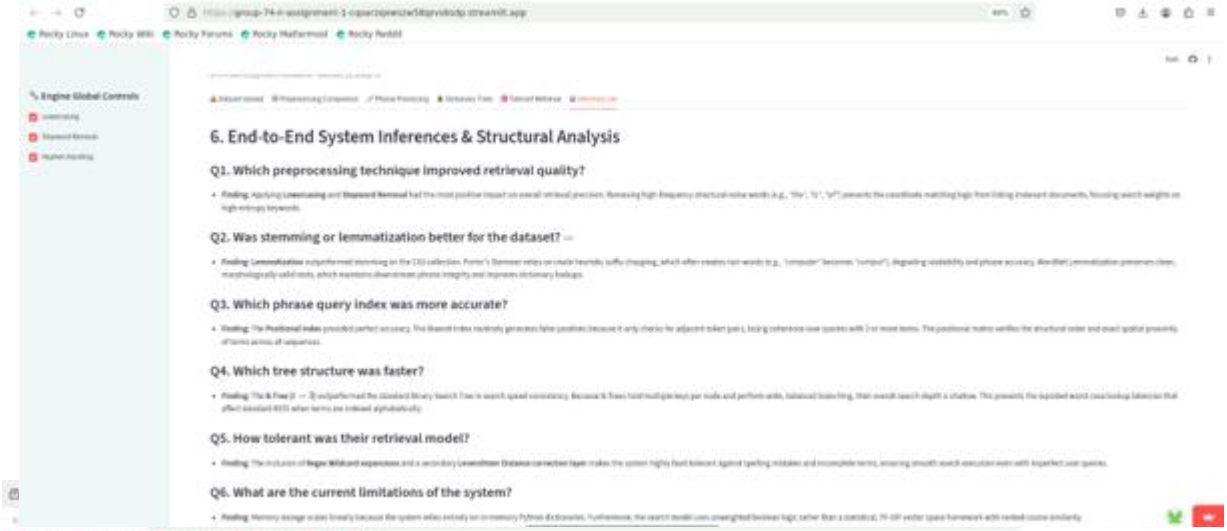
## Screenshots:

Screenshots of execution in virtual lab.

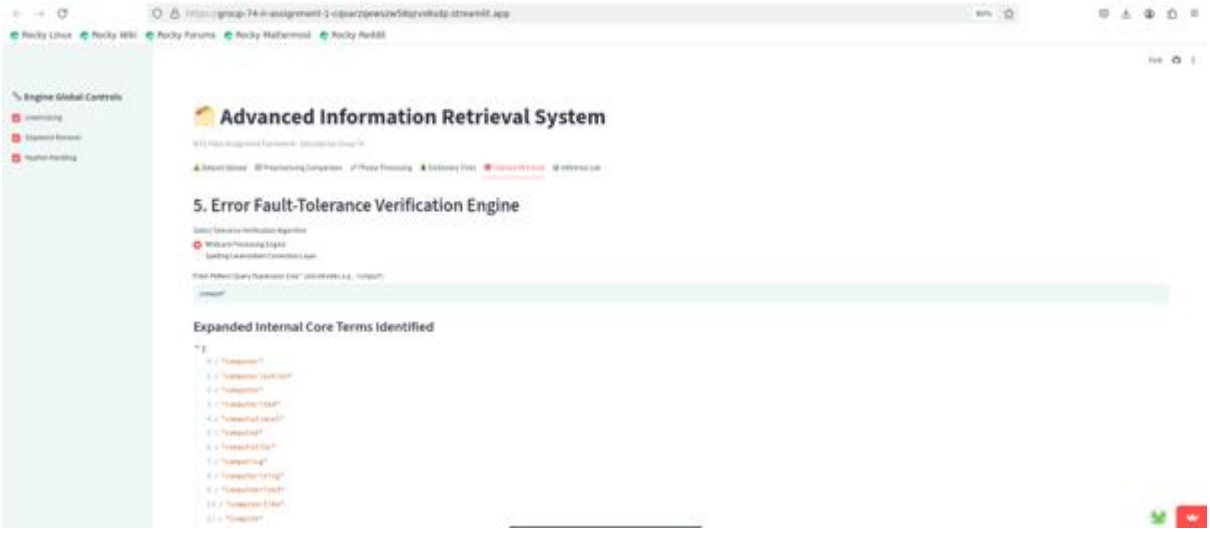


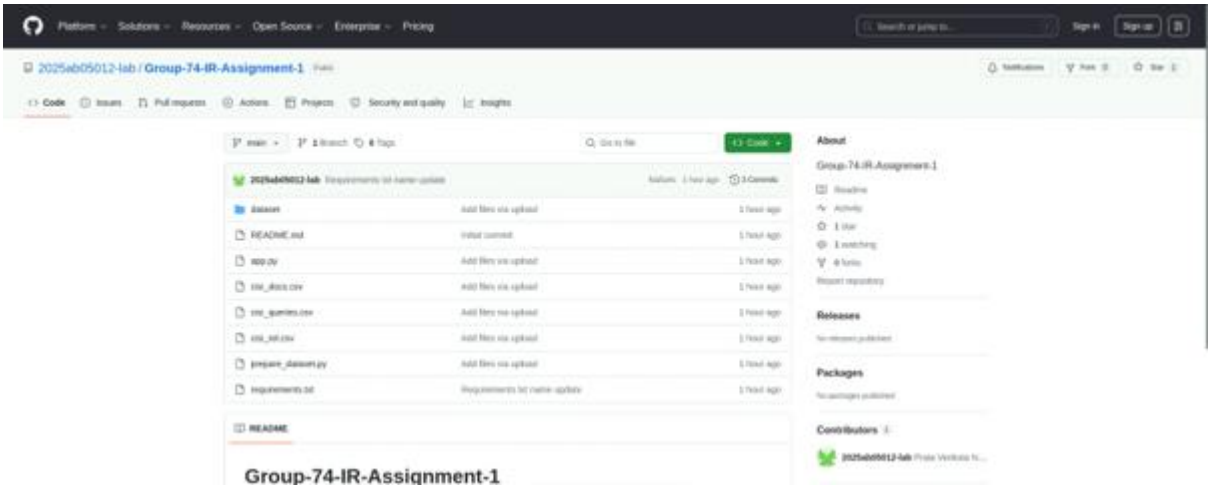




Engine Global Controls
6. End-to-End System Inferences & Structural Analysis
Q1. Which preprocessing technique improved retrieval quality?
Q2. Was stemming or lemmatization better for the dataset?
Q3. Which phrase query index was more accurate?
Q4. Which tree structure was faster?
Q5. How tolerant was their retrieval model?
Q6. What are the current limitations of the system?


Engine Global Controls
4. Performance Benchmarks: BST vs. B-Tree Dictionaries
Search Engine Latency Assessment Matrix


Engine Global Controls
Advanced Information Retrieval System
5. Error Fault-Tolerance Verification Engine
Expanded Internal Core Terms Identified


Platform
Solutions
Resources
Open Source
Enterprise
Pricing
Sign in
Sign up
2025ab05012-lab / Group-74-IR-Assignment-1
Code
Issues
Pull requests
Actions
Projects
Security and quality
Insights
main
Go to file
Code
dataset
README.md
app.py
irsi_docs.csv
irsi_queries.csv
irsi_tsf.csv
prepare_dataset.py
requirements.txt
About
Group-74-IR-Assignment-1
Releases
Packages
Contributors
README
Group-74-IR-Assignment-1